

1 Einleitung

In diesem Part der Arbeit, versuchen wir Musikstücke ihren Komponisten zuzuordnen. Dafür verwenden wir das Datenset "Classical Music MIDI as CSV", welches auf Kaggle zur Verfügung steht. Auch hier liegt bei den einzelnen Musikstücken wieder nicht die Audiodatei selbst vor, sondern ein MIDI File, mit welchem die Maschine umgehen kann.

Das Datenset ist wie folgt aufgebaut: Es gibt 10 verschiedene Kategorien, in welche die Musikstücke eingeordnet werden. Gibt es zu einem Künstler mehr als 100 Stücke, so hat dieser seine eigene Kategorie. Von vielen Künstlern stehen aber weniger als 100 Werke zur Verfügung. Diese (sowie auch die Stücke der Komponisten mit mehr als 100 Werken) finden sich in der Kategorie "ALL". Es gibt eine Kategorie "Varios - Título desconocido", was spanisch ist für "Verschiedene - Titel unbekannt", wo sich Stücke befinden, bei welchen weder Titel noch Künstler bekannt ist.

In diesem Datenset gibt es folgende Kategorien:

1. ALL, mit allen Stücken aus diesem Datenset,
2. Bach, wobei hier 35.191 Datenpunkte vorhanden sind,
3. Beethoven, mit 29.124,
4. Chopin, mit 6.328,
5. Händel, mit 6.031,
6. Mozart, mit 25.430,
7. Scarlatti, mit 12.335,
8. Schubert, mit 16.685,
9. Varios - Título desconocido und
10. Vivaldi, mit 7.494.

Künstler mit weniger als 100 Stücken sind beispielsweise: Pachelbel, Jakobowski oder Strauss. Über die Stücke selbst liegen wieder verschiedene Informationen, wie beispielsweise die Noten oder der Takt vor. Für die Klassifikation haben wir nur die Noten verwendet. Die Noten sind Zahlenwerte von eins bis 127, sprich es ist gleich aufgebaut, wie beim Teil der Generierung. Weitere Informationen zum Aufbau des Datensets, siehe (Vincentloos, 2022).

Im Allgemeinen haben wir in diesem Abschnitt die Kategorien 1 (ALL) und 8 (Título desconocido) ausgelassen, weil sie alle Stücke bzw. Stücke ohne Künstler beinhalten und wir das für die Klassifikation nicht verwenden können.

2 Vorbereitung der Daten

Kristian schreibst du das noch, weil sonst würd ich einfach schreiben:
"ist gleich wie bei Generierung"

3 Klassifikation mit RandomForestClassifier

3.1 Erster Versuch

Zur Klassifikation verwenden wir einen Random Forest. Nachdem wir die Daten vorbereitet haben, ordnen wir die Stücke ihren Komponisten zu. Da die Daten sehr unausgeglich sind (so hat zum Beispiel Bach am meisten mit 35.191 Daten und Händel mit nur 6.031 die wenigsten Informationen) muss das Datenset balanciert werden. Dabei gehen aber im schlimmsten Fall (von Bach) 29.160 Datenpunkte, also über 80% der Informationen verloren. Das ist natürlich nicht gut für die Klassifikation, wie sich später auch zeigen wird.

Trainiert man einen Random Forest mit den Default-Werten (scikit-learn developers, 2025a), so erhält man eine Accuracy von 27%. Also schlecht. Setzt man den Defaultwert von *n_estimators* (die Anzahl der Bäume im Forest) höher, so verbessert sich die Accuracy. Bei beispielsweise 1.000 Bäumen hat man eine Genauigkeit von 31%, was besser aber immernoch schlecht ist. Das Problem, wenn man *n_estimators* höher setzt, ist, dass es länger braucht zum Laden. Betrachtet man die Confusionmatrix (1), kann man folgendes erkennen:

- Stücke von zum Beispiel Schubert können gut klassifiziert werden.
- Zu großen Verwechslungen kommt es beispielsweise zwischen Händel und Bach.
- Im Allgemeinen sieht man aber, dass Musikstücke oft in der gleichen Zeitepoche, in welcher die Künstler gelebt haben, vertauscht werden.

Durch die Beobachtung, dass Werke, die der gleichen Epoche angehören öfter verwechselt werden, sind wir auf die Idee gekommen im zweiten Versuch über zwei (bzw. dann fünf) Modelle zu klassifizieren. Hier haben wir das Musikstück zuerst seiner Epoche und dann innerhalb dieser, dem Komponisten zugeordnet.

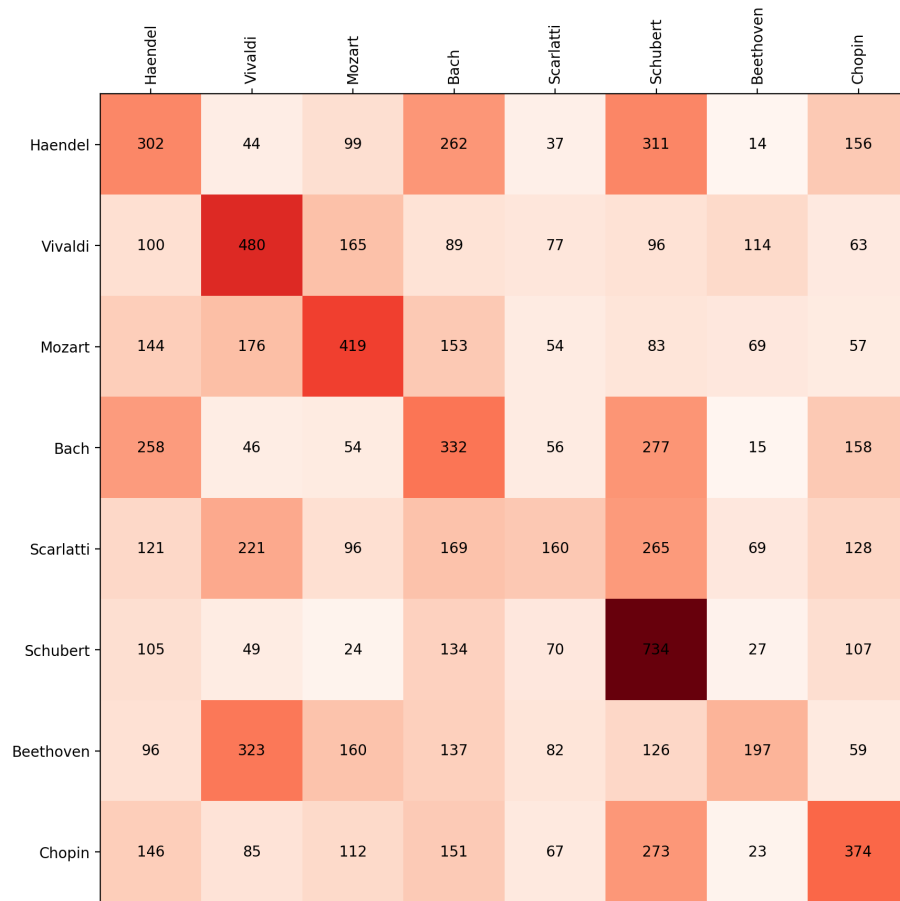


Figure 1: Klassifikation mit 1.000 Bäumen im Forest

3.2 Zweiter Versuch: Mit Einbezug der zeitlichen Epochen

3.2.1 Modell zur Klassifizierung in die Epochen

Wir trainieren zuerst nach den verschiedenen Epochen, in welchen die Künstler gelebt haben. Diese sind:

Barock : Scarlatti, Vivaldi, Bach, Händel, mit 61.051 Datenpunkten (Kurzstueckmeister, 2026a),

Klassik : Mozart, Beethoven, mit 54.554 (Kurzstueckmeister, 2026b) und

Romantik : Schubert, Chopin, mit 23.013 (Thornard, 2026).

Die Accuracy beim Einordnen der Stücke in ihre jeweiligen Epochen beträgt 55%. Auch hier verlieren wir wieder viele Daten beim Balancieren, da bei der

Zuordnung zu den verschiedenen Zeitabschnitten grosse Unterschiede auftreten. So hat die Romantik nur noch 1/3 der Daten, die beim Barock vorliegen. Dennoch ist das Problem zu Versuch eins, wo wir 80% verloren haben, besser geworden.

3.2.2 Modelle zum Klassifizieren innerhalb der einzelnen Epochen

Innerhalb der verschiedenen Epochen (Barock, Klassik und Romantik) werden die Werke ihren Komponisten zugeordnet. Das hat, wie man in der Grafik 2 erkennen kann vor allem in der Klassik sowie Romantik gut funktioniert. Dort erreichen wir eine Accuracy von 67% bzw. 66%. Nur im Barock funktioniert es schlechter mit einer Genauigkeit von 43%, wobei vor allem Scarlatti und Vivaldi oft falsch eingeordnet werden.

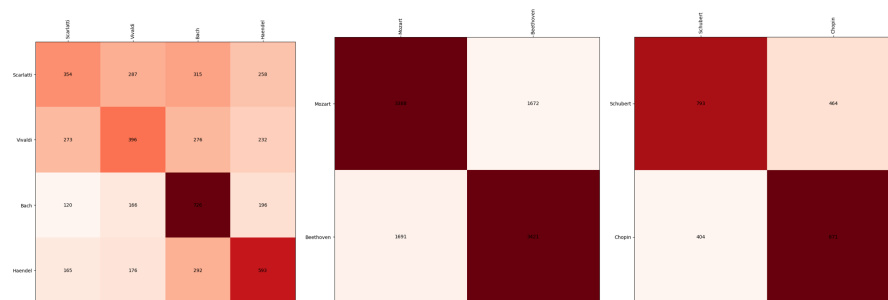


Figure 2: Links: Barock, Mitte: Klassik, Recht: Romantik

3.2.3 Verbinden der Modelle

Schlussendlich haben wir diese vier Modelle in folgendem Sinne verbunden:

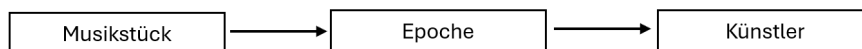


Figure 3: Verbinden der Modelle

Wie man aus dem Flowchart 3 entnehmen kann, wird ein Musikstück genommen und seiner Epoche zugeordnet. Innerhalb dieser Epoche wird es dann wieder seinem Künstler zugeteilt. Mit diesen fünf Modellen haben wir eine Accuracy von knapp 40% erreicht, was etwas besser ist, als zuvor. Dennoch ist sie nicht besonders gut, was daran liegt, dass schon beim Einteilen in die verschiedenen Epochen nur eine Genauigkeit von 55% erreicht wird. Somit wäre das dann auch die maximal mögliche Genauigkeit.

4 Klassifizieren mittels Support Vector Classification (SCV)

In einem neuen Versuch verwenden wir anstatt des RandomForestClassifiers eine Support Vektor Maschine, mit welcher wir dann Klassifizieren (gennant auch SVC für Support Vector Classification). Bei der SVC werden die Objekte in Klassen unterteilt, wobei diese durch Vektoren im Vektorraum dargestellt werden. Der Algorithmus versucht dann, die Objekte durch Hyperebenen so zu trennen, dass der Abstand der Vektoren, welche am nächsten an der Hyperebene liegen maximiert wird. Gibt man dem Modell später neue unbekannte Daten, versucht es, diese in die vorgelernten Klassen einzuteilen, anhand der Trennlinien, die zuvor antrainiert wurden (Bitbert, 2025).

Da dies aber vorallem für viele Daten sehr aufwändig ist, haben wir die Daten pro Künstler nur auf 603 (also 1/10 von dem, was wir aufgrund des Balancieren verwenden könnten) beschränkt. Ohne diese Maßnahme wäre es aufgrund der Ladedauer nicht möglich gewesen, einen RandomizedSearchCV anzuwenden, welcher die besten Parameter für die Support Vector Classification sucht.

Mit den Standardparametern der Support Vektor Klassifizierung (scikit-learn developers, 2025b) erhält man eine Accuracy von nur 18%.

Die Suche nach den besten Parametern ergibt eine Genauigkeit von 23%. Diese Parameter sind:

C : 0.5,

Kernel : Poly, (Diesen Parameter braucht man, da die SVC normalerweise nur linear sein würde. Das Modell versucht also die Daten linear zu trennen. Dies ist aber nicht möglich, da die gegeben Daten nicht linear sind. Also verwendet man einen Kernel, um auch gekrümmte Schnittlinien zu erhalten.)

Degree : 3,

Gamma : auto, (Diesen Koeffizienten braucht man, wenn der Kernel "rbf", "poly" oder "sigmoid" ist. Wir haben aber nur zwischen Poly, Sigmoid und Linear den besten Kerneltyp gesucht. Somit würde γ nur bei Poly oder Sigmoid verwendet werden.)

Tol : 0.01.

Dementsprechend ist die beste Methode für eine Klassifizierung, über die Zeitepochen die Stücke einzuteilen und dann innerhalb dieser nochmals spezifisch die Künstler zuzuordnen. Jedoch ist auch dieses nicht gut, da nur eine Genauigkeit von etwa 40% erreicht werden kann. Das Problem liegt dabei daran, dass sich die Stücke von der Abfolge der Noten (der Melodie) nicht weit genug unterscheiden, dass beim Trainieren ein Unterschied erkannt wird.